Praktikum 3

Thema: Vererbung

Aufgabe 1: Einstiegsaufgabe: Fahrzeuge (30 Minuten)

In dieser Aufgabe soll das Prinzip der Vererbung in Java umgesetzt werden.

1. Erstelle eine Basisklasse `Fahrzeug` mit folgenden Eigenschaften:
   - `marke` (`String`)
   - `geschwindigkeit` (`int`)
2. Implementiere:
   - Einen Konstruktor zur Initialisierung der Attribute
   - Eine Methode `beschleunigen(int wert)`, die die Geschwindigkeit erhöht
   - Eine Methode `anzeigen()`, die alle Informationen ausgibt
3. Erstelle zwei abgeleitete Klassen:
   - `Auto`
     - zusätzliches Attribut: `anzahlTueren (int)`
   - `Fahrrad`
     - zusätzliches Attribut: `hatKlingel (boolean)`
4. Beide Klassen sollen:
   - den Konstruktor der Basisklasse verwenden
   - die Methode `anzeigen()` überschreiben und ihre eigenen Attribute zusätzlich ausgeben
5. Schreibe in einer Hauptklasse `Main` eine `main`-Methode, in der:
   - ein Auto und ein Fahrrad erzeugt werden
   - beide beschleunigt werden
   - anschließend ihre Daten ausgegeben werden

Aufgabe 2: Überschreiben und Überladen (30 Minuten)

In dieser Aufgabe wird Vererbung, Methodenüberschreibung und Konstruktorüberladung kombiniert.

1. Erstelle eine Basisklasse `Mitarbeiter` mit folgenden Attributen:
   - name (String)
   - gehalt (double)
2. Implementiere in `Mitarbeiter`:
   - zwei Konstruktoren:
     - einen Konstruktor mit Parametern (`name, gehalt`)
     - einen Konstruktor nur mit `name`, der ein Standardgehalt von 3000 setzt
   - Eine Methode `berechneBonus()`, die standardmäßig 10% des Gehalts zurückgibt
   - Eine Methode `anzeigen()`, die alle Informationen ausgibt
3. Erstelle eine abgeleitete Klasse **Manager**, die von Mitarbeiter erbt:
   - zusätzliches Attribut: abteilung (`String`)
4. Implementiere in Manager:
   - Zwei Konstruktoren mit den Parametern:
     - (`name, gehalt, abteilung`)
     - (`name, abteilung`) → verwendet Standardgehalt
   - Überschreibe die Methode `berechneBonus()`:
     - Manager erhalten 20% Bonus
   - Überschreibe die Methode `anzeigen()`, sodass zusätzlich die Abteilung ausgegeben wird
5. Schreibe in einer Hauptklasse `Main` eine `main`-Methode:
   - Erzeuge zwei Mitarbeiter-Objekte mit unterschiedlichen Konstruktoren
   - Erzeuge zwei Manager-Objekte mit unterschiedlichen Konstruktoren
   - Gib deren Daten inklusive Bonus aus

Aufgabe 3: Polymorphie (30 Minuten)

In dieser Aufgabe geht es um Polymorphie mithilfe eines Arrays der Oberklasse.

1. Erstelle eine Basisklasse `Form` mit:
   - Attribut: name (`String`)
   - Konstruktor zur Initialisierung
   - Methode `berechneFlaeche()`, die standardmäßig 0 zurückgibt
   - Methode `anzeigen()`, die den Namen ausgibt
2. Erstelle zwei abgeleitete Klassen:
   - `Rechteck`
     - Attribute: `breite, hoehe`
   - `Kreis`
     - Attribut: `radius`
3. Beide Klassen sollen:
   - den Konstruktor der Oberklasse verwenden
   - die Methode `berechneFlaeche()` **überschreiben**:
     - Rechteck ⟶ Fläche = Breite × Höhe
     - Kreis ⟶ Fläche = $\pi \times r^2$
   - optional die `anzeigen()`-Methode erweitern
4. In der `main`-Methode der `Main`-Klasse:
   - Erstelle ein Array für alle Arten von Formen
   - Speichere darin verschiedene Objekte (Rechteck, Kreis)
   - Durchlaufe das Array und gib für jedes Objekt den Namen und die berechnete Fläche aus

Zusatzaufgabe:

Statt der `anzeigen()`-Methode könnte man bei allen 3 Aufgaben auch die `toString()`-Methode (geerbt von `Object`) überschreiben. Damit könnte man die Objekte direkt per `System.out.println(ihreVariableMitObjekt)` ausdrucken. Probieren Sie es aus!